

IT342 Refactoring Report

Project: Knock Knock

Student: Niña Nicole S. Villadarez

Course: IT342

1. Creational Pattern: Factory Method

Applied Design Pattern

- Pattern Name: Factory Method
- Where it was applied: NotificationFactory.java and VisitServiceImpl.java.

Before vs. After Description

- Original Implementation: The VisitServiceImpl manually instantiated Notification objects using constructors and multiple set methods inside various service methods (check-in, check-out, etc.).

```
Windsurf: Refactor | Explain | Generate Javadoc | X
private void createVisitNotification(Visit visit, String eventType) {
    if (visit == null || visit.getVisitor() == null) {
        return;
    }

    String condoName = (visit.getCondo() != null && visit.getCondo().getName() != null)
        ? visit.getCondo().getName()
        : "your condominium";

    String unitNumber = visit.getUnitNumber();
    String unitLabel = (unitNumber != null && !unitNumber.isBlank())
        ? "Unit " + unitNumber
        : "your unit";

    LocalDate date = visit.getVisitDate();
    String formattedDate = null;
    if (date != null) {
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("MMMM d, yyyy");
        formattedDate = date.format(formatter);
    }

    String type;
    String title;
    String message;

    switch (eventType) {
        case "check_in":
            type = "check_in";
            title = "Welcome!";
            message = "You've successfully checked into " + unitLabel + " at " + condoName + ". Enjoy your visit!";
            break;
        case "check_out":
            type = "check_out";
            title = "Check-out complete";
```

- Problems: Tight Coupling. The logic for "what a notification looks like" was scattered across the service. If the notification format changed, I had to edit every service method.

Justification

- Why choose this pattern: The Factory Method centralizes the creation logic.
- Improvement: It ensures that all notifications (Scheduled, Missed, Checked-in) follow a consistent structure while keeping the VisitService clean.

```

J Visit.java J VisitServiceImpl.java 1, M J NotificationFactory.java U X J VisitController.java J SupabaseConfig.java
backend > src > main > java > com > knockknock > backend > service > J NotificationFactory.java > ...
14 @Component
15 public class NotificationFactory {
16
17     private static final DateTimeFormatter DATE_FORMATTER =
18         DateTimeFormatter.ofPattern("MMMM d, yyyy");
19
20     public Notification createVisitNotification(Visit visit, String eventType) {
21         if (visit == null || visit.getVisitor() == null) {
22             return null;
23         }
24
25         String condoName = (visit.getCondo() != null && visit.getCondo().getName() != null)
26             ? visit.getCondo().getName()
27             : "your condominium";
28
29         String unitNumber = visit.getUnitNumber();
30         String unitLabel = (unitNumber != null && !unitNumber.isBlank())
31             ? "Unit " + unitNumber
32             : "your unit";
33
34         LocalDate date = visit.getVisitDate();
35         String formattedDate = null;
36         if (date != null) {
37             formattedDate = date.format(DATE_FORMATTER);
38         }
39
40         String type;
41         String title;
42         String message;
43
44         switch (eventType) {
45             case "check_in":
46                 type = "check_in";
47                 title = "Welcome!";
48                 message = "You've successfully checked into " + unitLabel + " at " + condoName + ". Enjoy your visit!";
49                 break;

```

```

Windsurf: Refactor | Explain | Generate Javadoc | X
private void createVisitNotification(Visit visit, String eventType) {
    Notification notification = notificationFactory.createVisitNotification(visit, eventType);
    if (notification != null) {
        notificationRepository.save(notification);
    }
}

```

2. Creational Pattern: Builder

Applied Design Pattern

- Pattern Name: Builder
- Where it was applied: Visit.java and Notification.java.

Before vs. After Description

- Original Implementation: Creating a Visit required a constructor with 8+ parameters or a long list of .set() calls.

```
public class VisitServiceImpl implements VisitService {  
    Windsurf: Refactor | Explain | Generate Javadoc | X  
    @Override  
    public Visit createVisit(UUID visitorId, UUID condoId, String unitNumber, String dateOfVisit,  
        String purposeOfVisit) {  
        User visitor = userRepository.findById(visitorId)  
            .orElseThrow(() -> new RuntimeException(HttpStatus.NOT_FOUND, reason: "Visitor not found"));  
  
        Condo condo = condoRepository.findById(condoId)  
            .orElseThrow(() -> new RuntimeException(HttpStatus.NOT_FOUND, reason: "Condominium not found"));  
  
        // Parse the date string to LocalDate (format: YYYY-MM-DD)  
        LocalDate parsedDate = LocalDate.parse(dateOfVisit);  
  
        // Generate reference number with condo code  
        String referenceNumber = generateReferenceNumber(condo.getCode());  
  
        Visit visit = new Visit(visitor, condo, referenceNumber, unitNumber, purposeOfVisit, parsedDate, status: "SCHEDULED");  
        Visit saved = visitRepository.save(visit);  
  
        // Notify visitor about scheduled visit  
        createVisitNotification(saved, eventType: "scheduled");  
  
        return saved;  
    }  
}
```

- Problems: Readability and Safety. It was easy to mix up parameters (like putting the Unit Number where the Condo ID goes) because they are all Strings.

Justification

- Why choose this pattern: The Builder pattern provides a readable, chainable way to create complex objects.
- Improvement: It ensures required fields are set before the object is "built" and prevents parameter mix-ups.

```
d > src > main > java > com > knockknock > backend > service > J VisitServiceImpl.java > VisitServiceImpl  
public class VisitServiceImpl implements VisitService {  
    public Visit createVisit(UUID visitorId, UUID condoId, String unitNumber, String dateOfVisit,  
  
        Visit visit = Visit.builder()  
            .visitor(visitor)  
            .condo(condo)  
            .referenceNumber(referenceNumber)  
            .unitNumber(unitNumber)  
            .purpose(purposeOfVisit)  
            .visitDate(parsedDate)  
            .status(status: "SCHEDULED")  
            .build();  
        Visit saved = visitRepository.save(visit);  
  
        // Notify visitor about scheduled visit  
        createVisitNotification(saved, eventType: "scheduled");  
  
        return saved;  
    }  
}
```

3. Structural Pattern: Adapter

Applied Design Pattern

- Pattern Name: Adapter
- Where it was applied: StorageClient.java (Interface) and SupabaseStorageAdapter.java.

Before vs. After Description

- Original Implementation: The FileStorageService was hardcoded to use OkHttpClient to talk specifically to the Supabase API.

```
public FileStorageService(SupabaseConfig supabaseConfig) {  
    this.supabaseConfig = supabaseConfig;  
    this.httpClient = new OkHttpClient();  
}
```

- Problems: Vendor Lock-in. If the project moves from Supabase to AWS S3 or Google Cloud Storage, the entire service would need a rewrite.

Justification

- Why choose this pattern: The Adapter hides the Supabase-specific details behind a generic StorageClient interface.
- Improvement: The rest of the app only knows how to upload() or delete(), making it vendor-agnostic.

```
private final StorageClient storageClient;  
  
public FileStorageService(StorageClient storageClient) {  
    this.storageClient = storageClient;  
}  
  
/**  
 * Upload a file using the configured storage client and return the public URL.  
 */  
Windsurf: Refactor | Explain | X  
public String uploadFile(MultipartFile file, String folderPath) throws IOException {  
    System.out.println("[FileStorageService] Delegating upload to StorageClient");  
    return storageClient.uploadFile(file, folderPath);  
}  
  
/**  
 * Delete a file using the configured storage client.  
 */  
Windsurf: Refactor | Explain | X  
public void deleteFile(String filePath) {  
    System.out.println("[FileStorageService] Delegating delete to StorageClient");  
    try {  
        storageClient.deleteFile(filePath);  
    } catch (IOException e) {  
        System.err.println("[FileStorageService] Error deleting file via StorageClient: " + e.getMessage());  
        e.printStackTrace();  
    }  
}
```

```
public interface StorageClient {

    /**
     * Upload a file to the underlying storage provider and return a public URL.
     */
    Windsurf: Refactor | Explain | ✕
    String uploadFile(MultipartFile file, String folderPath) throws IOException;

    /**
     * Delete a file from the underlying storage provider.
     */
    Windsurf: Refactor | Explain | ✕
    void deleteFile(String filePath) throws IOException;
}
```

```
public class SupabaseStorageAdapter implements StorageClient {

    private final SupabaseConfig supabaseConfig;
    private final OkHttpClient httpClient;

    public SupabaseStorageAdapter(SupabaseConfig supabaseConfig) {
        this.supabaseConfig = supabaseConfig;
        this.httpClient = new OkHttpClient();
    }
}
```

4. Behavioral Pattern: Observer

Applied Design Pattern

- Pattern Name: Observer
- Where it was applied: VisitStatusChangedEvent.java and NotificationListener.java.

Before vs. After Description

- Original Implementation: When a visit status changed (e.g., checking in), the VisitService had to manually call the NotificationService and the VisitStatusHistoryRepository.

```
Windsurf: Refactor | Explain | Generate Javadoc | X
@Override
public Visit checkInVisit(UUID visitId, UUID adminCondoId, String modifiedByName, String modifiedByRole) {
    Visit visit = visitRepository.findById(visitId)
        .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND, reason: "Visit not found"));

    String previousStatus = visit.getStatus();

    if (!"SCHEDULED".equalsIgnoreCase(visit.getStatus())) {
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST, reason: "Visit is not in SCHEDULED status");
    }

    if (visit.getCondo() == null || visit.getCondo().getCondoId() == null ||
        !visit.getCondo().getCondoId().equals(adminCondoId)) {
        throw new ResponseStatusException(HttpStatus.FORBIDDEN, reason: "Visit does not belong to this condominium");
    }

    visit.setStatus(status: "CHECKED-IN");
    visit.setCheckInTime(LocalDateTime.now());
    Visit saved = visitRepository.save(visit);

    recordStatusChange(saved, previousStatus, saved.getStatus(), modifiedByName, modifiedByRole);

    // Notify visitor about check-in
    createVisitNotification(saved, eventType: "check_in");

    return saved;
}
```

```
Windsurf: Refactor | Explain | Generate Javadoc | X
@Override
public Visit checkOutVisit(UUID visitId, UUID adminCondoId, String modifiedByName, String modifiedByRole) {
    Visit visit = visitRepository.findById(visitId)
        .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND, reason: "Visit not found"));

    String previousStatus = visit.getStatus();

    if (!"CHECKED-IN".equalsIgnoreCase(visit.getStatus())) {
        throw new ResponseStatusException(HttpStatus.BAD_REQUEST, reason: "Visit is not in CHECKED-IN status");
    }

    if (visit.getCondo() == null || visit.getCondo().getCondoId() == null ||
        !visit.getCondo().getCondoId().equals(adminCondoId)) {
        throw new ResponseStatusException(HttpStatus.FORBIDDEN, reason: "Visit does not belong to this condominium");
    }

    visit.setStatus(status: "CHECKED-OUT");
    visit.setCheckOutTime(LocalDateTime.now());
    Visit saved = visitRepository.save(visit);

    recordStatusChange(saved, previousStatus, saved.getStatus(), modifiedByName, modifiedByRole);

    // Notify visitor about check-out
    createVisitNotification(saved, eventType: "check_out");

    return saved;
}
```

- Problems: Violation of Separation of Concerns. The VisitService was doing too many things: updating the visit, logging history, and sending alerts.

Justification

- Why choose this pattern: Using Spring's `@EventListener` implements the Observer pattern effectively.
- Improvement: The VisitService now only updates the status and "publishes" an event, decoupling it from side effects like logging or notifications.

```
public class NotificationListener {  
    Windsurf: Refactor | Explain | Generate Javadoc | X  
    @EventListener  
    public void onVisitStatusChanged(VisitStatusChangedEvent event) {  
        Visit visit = event.getVisit();  
  
        // 1. Record status change history  
        VisitStatusHistory history = new VisitStatusHistory();  
        history.setVisit(visit);  
        history.setPreviousStatus(event.getPreviousStatus());  
        history.setNewStatus(event.getNewStatus());  
        history.setModifiedByName(event.getModifiedByName());  
        history.setModifiedByRole(event.getModifiedByRole());  
        history.setChangedAt(LocalDate.now());  
        visitStatusHistoryRepository.save(history);  
  
        // 2. Send notification to visitor  
        Notification notification = notificationFactory.createVisitNotification(visit, event.getEventType());  
        if (notification != null) {  
            notificationRepository.save(notification);  
        }  
    }  
}
```